Digital Systems

Lecture 2

State and Instructions

Mark van der Wilk

Hilary Term 2025

UNIVERSITY OF OXFORD

Department of COMPUTER SCIENCE

Following the course of past years by Mike Spivey

❓ How do instructions alter the CPU state?

- Understand the effect of instructions on the state of the CPU.
- Worked example of executing an instruction.

❓ How are instructions encoded electronically?

And how does this affect programming the machine?

💪 Decode machine code

I.e. how to take machine code, and determine what it will do.

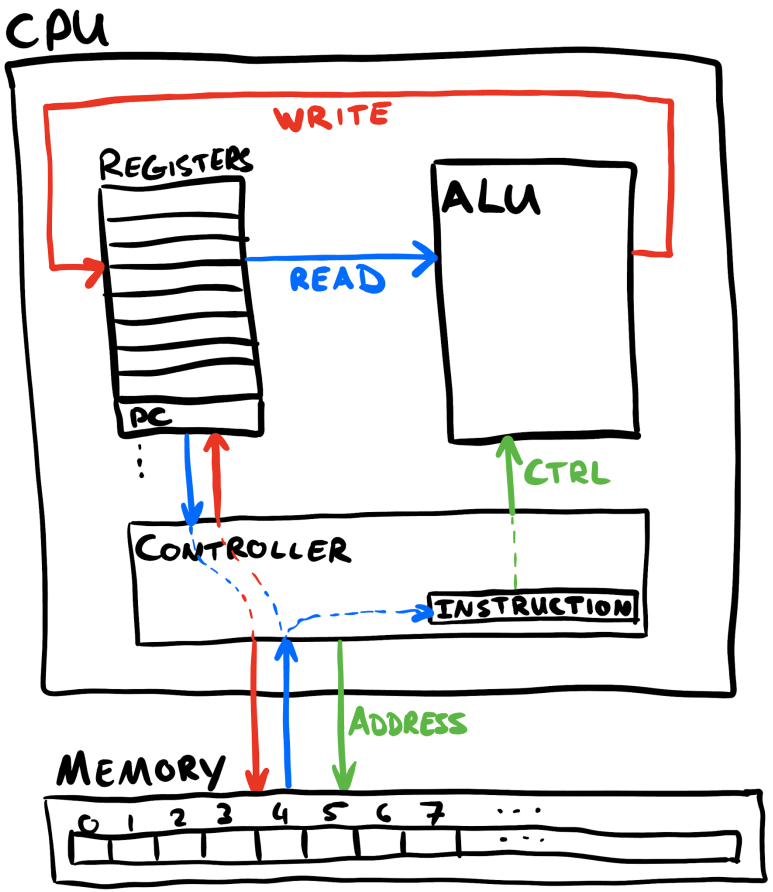
❓ How can we make parts of a program reusable?

How do subroutines work?

RISC + The von Neumann Architecture

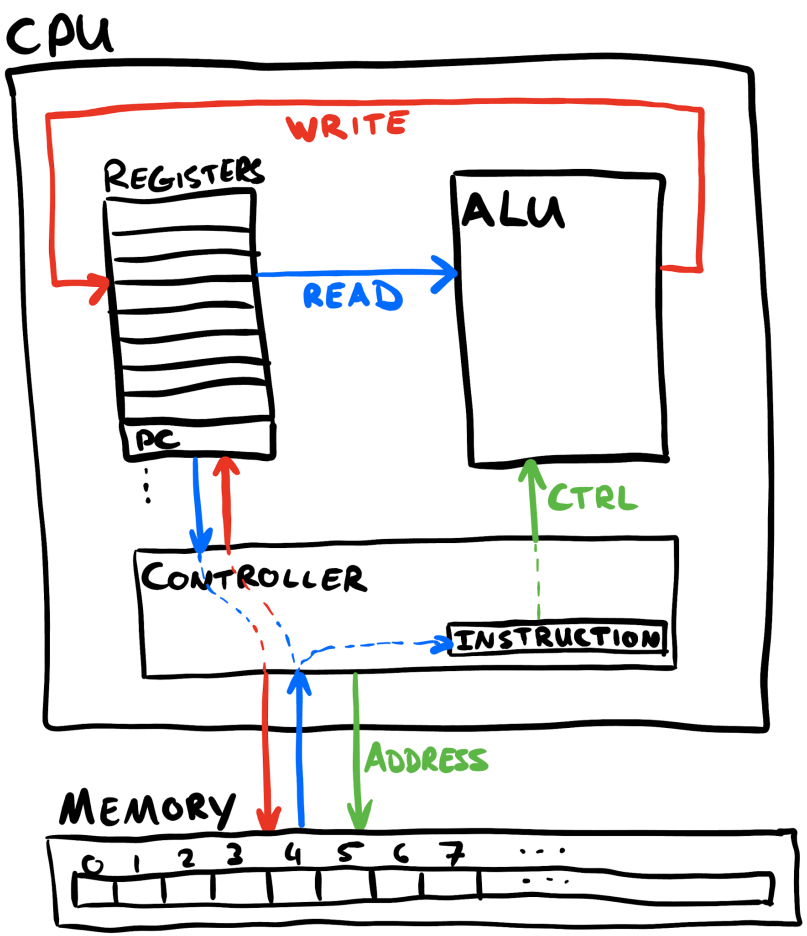
Many other architectures are provably equally powerful to Turing machine (if given infinite memory)

- State machine: CPU
  - State stored in "registers"
- Replace tape with **addressable memory**
- Conceptual separation between instructions and data
  - Can think of instructions as parameterising the lookup.



Operation

1. Controller fetches an **instruction** from memory, at location in Program Counter (pc) (What is instruction? Look up-table)
2. Decode instruction
   RISC, so instructions are:
   - Load / Store
     (register ↔ memory)
   - Arithmetic / Logic
     (register ↔ register)
3. Execute instruction (change state)
4. Increment pc by 1 instruction, and repeat



Implementation

We will discuss electronics more later, but for now:

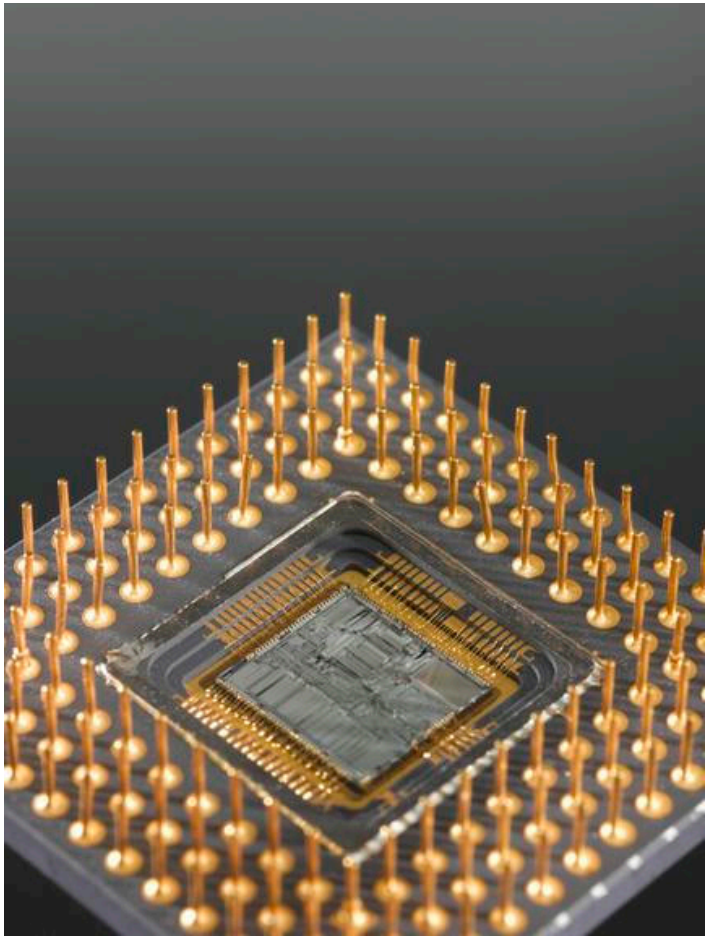
- All state is represented by an electrical voltage in a wire.
- Voltage is “high” or “low”, represented symbolically as “1” or “0”.
- Collection of wires encodes a binary number. # of wires is “width”.

We talk about “state”, and “registers”, and all that, but it’s important to keep in mind that…

> ⓘ **Symbols we use map onto things that are *physically real***
>
> If you could open up the chip, and attach an LED circuit to the wires, you would literally see them light up according to binary.

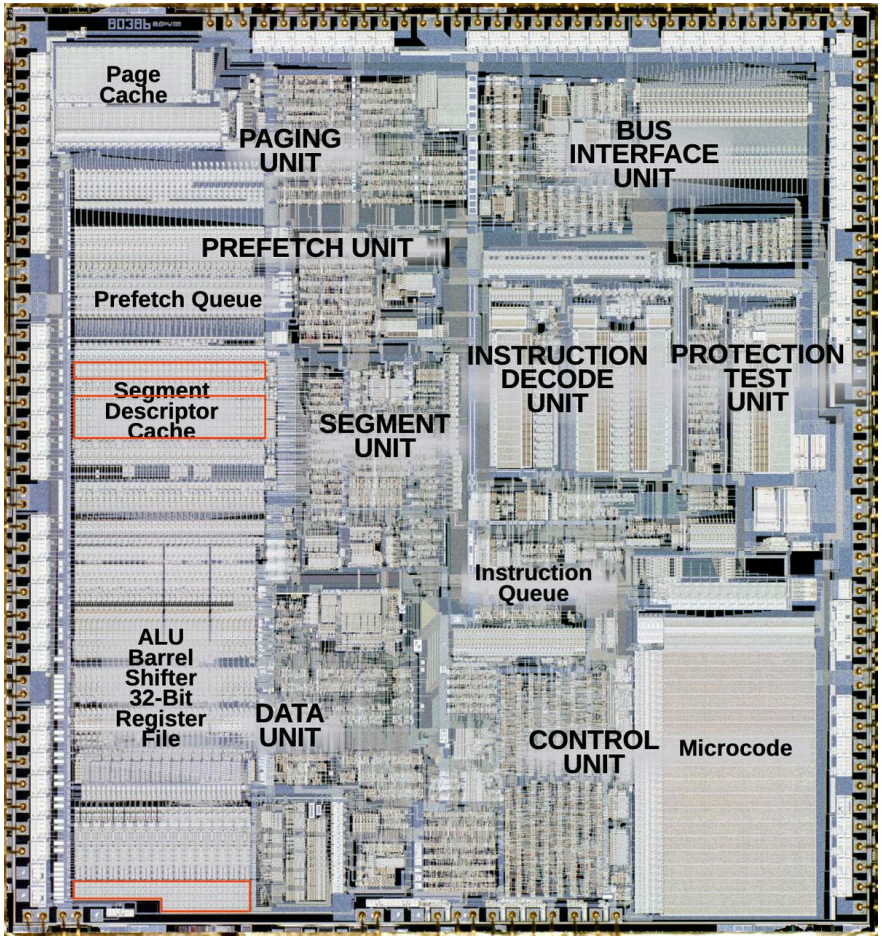
Implementation

- Electrical circuits are etched onto silicon.
- Can zoom in and *look* at circuits!
- Register file is a specific segment
- *Representation* of numbers, is high/low voltage on wires



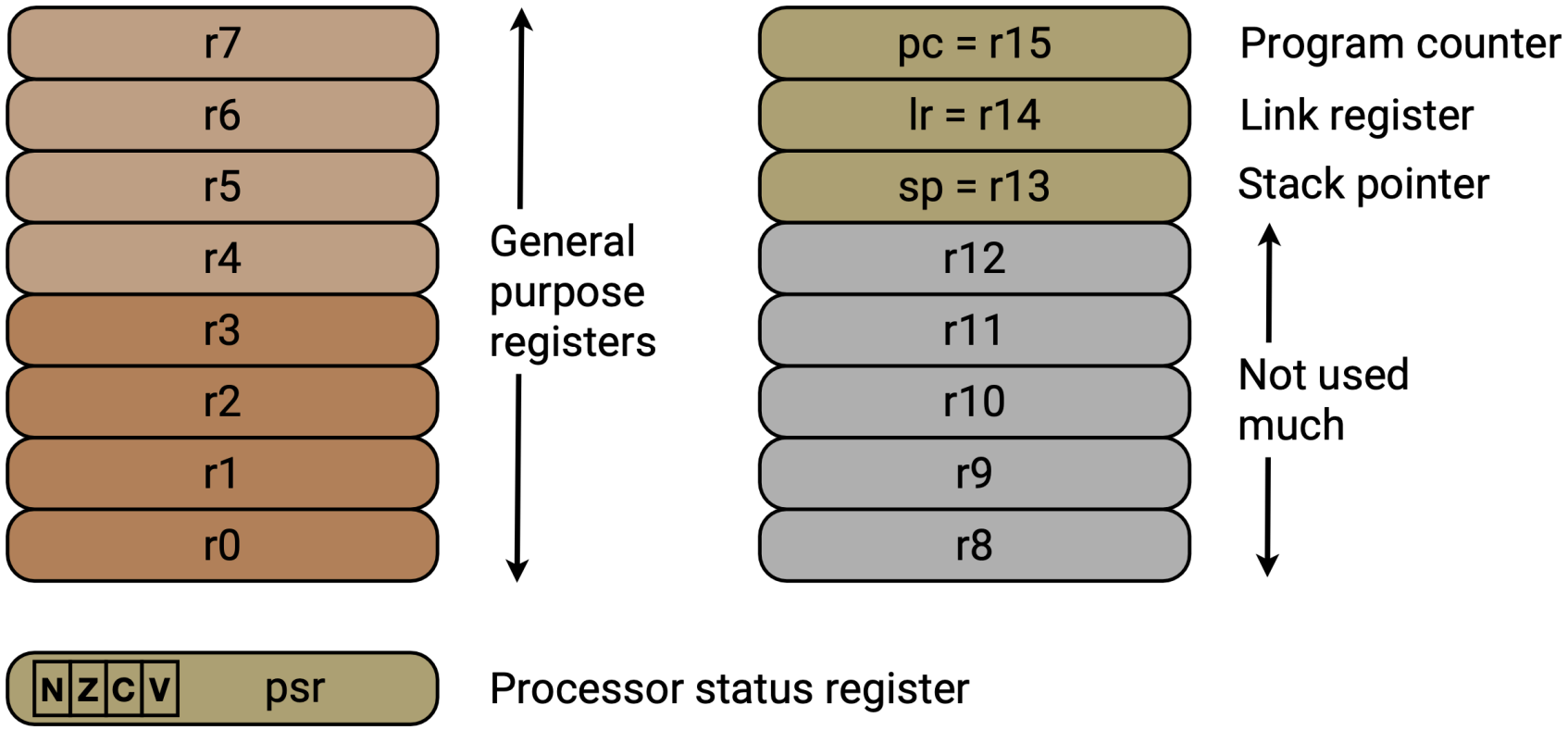
Implementation

- Electrical circuits are etched onto silicon.
- Can zoom in and *look* at circuits!
- Register file is a specific segment
- *Representation* of numbers, is high/low voltage on wires



Registers: A Closer Look

Instructions manipulate the state of the CPU, stored in **registers**.

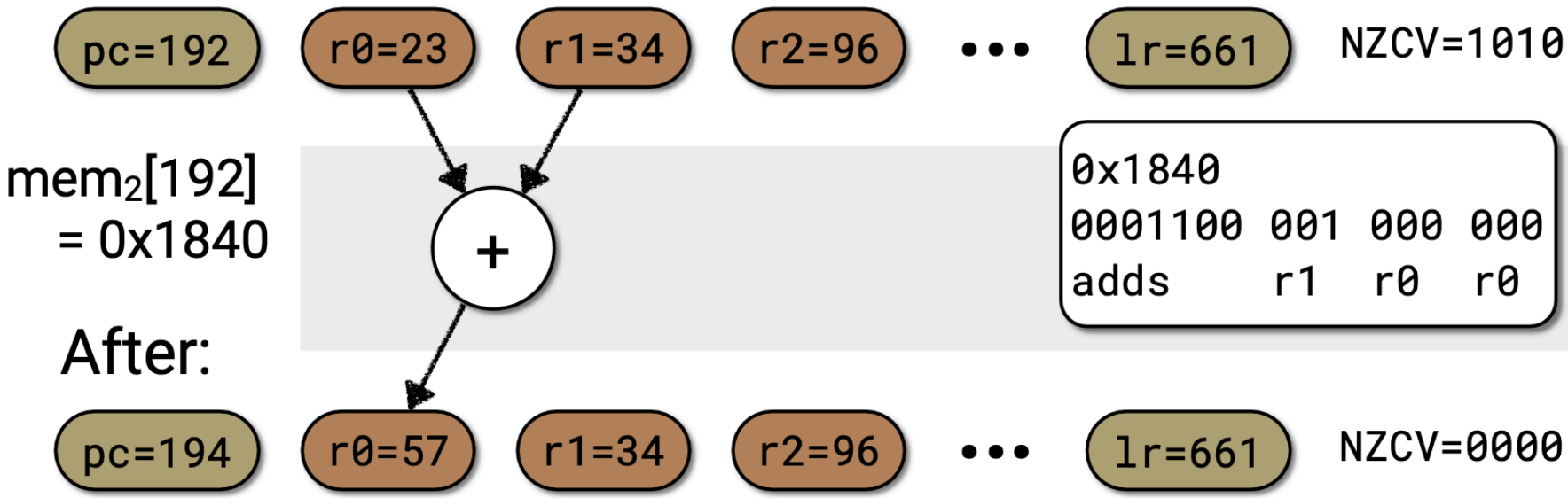


Registers: Programming Limitations & Convention

All Registers are 32 bits "wide" (How does this look in a physical circuit?)

- `r0 - r7` : General purpose registers. Can be used for any value used in any calculation.
- `r0 - r3` : Handled differently during subroutine calls.
- `r8 - r12` : Can be used for any value, but not all instructions can use them, so not as "general purpose" as `r0 - r7`.
- `pc` : Program counter. Stores memory address of next instruction.
- `lr` : Link register. Stores address where current subroutine returns.
- `sp` : Stack Pointer.
- `psr`: Processor Status Register. Different bits have very specific meanings. Some instructions write bits, some read bits.

The Effect of an Instruction



- Hex is convenient: Each hex digit corresponds to 4 bits.
- `0x1840 = 0b0001100001000000`
- In assembly language: `adds r0, r0, r1`.

Positional Number Systems

- We represent numbers as strings of digits, where each digit is a symbol taken from some set.
- Digits in decimal numbers come from a set with 10 elements.
- We can also construct numbers using sets of different size.

$$\mathrm{int}(s, b) = \sum_{i=0}^{\mathrm{len}(s)} d_i \cdot b^i$$

$$x = \mathrm{int}([d_{L-1}, ..., d_3, d_2, d_1, d_0])$$

💪 **Converting between different bases, e.g. binary, hexadecimal.**

Exercise: Write a little program to print a number in an arbitrary base!

Status Bits

The status bits `N`, `Z`, `C`, `V` are in `psr`. The instruction `adds` is does **`add`**ition, and sets the **`s`**tatus bits.

- `N`: Set if the result is negative. But how do we represent negative nums?
- `Z`: Set if the result is zero.
- `C`: Look at later.
- `V`: Look at later.

Assembly Language

- First level of abstraction up from binary machine code.
- There is a simple map between assembly and machine code.
- Basically machine code in human-readable form, with tiny bits of "syntactic sugar".
- Program that translates from assembly to machine code: *Assembler*.
- Given close correspondence, assemblers are simple programs, particularly when compared to compilers.

Assembly Language has… “Quirks”

Consider the instruction

- ✅ `adds r0, r1, #3`
- ✅ `adds r0, r0, #10`
- ❌ `adds r0, r1, #10`

Very annoying!

- Exam won’t test whether you can memorise these rules (you can look them up)
- But we do want to understand why this happens!

💪 Decoding machine code

I.e. how to take machine code, and determine what assembly instruction it maps to.

- As an assembly level programmer, no need to know this.
- … unless you want to understand *why* assembly works the way it does
- But as a *machine code* programmer, you definitely do!
  - Or as a designer of the underlying logic circuits!
  - Or if you want to *write* an assembler.

Rainbow Chart

In GitLab you can find `rainbow-chart.pdf`.

- Nice summary of how binary patterns map to instructions.
- List of all instructions used in the course.
- Will be included in the exam, if needed.

[A] Bits [15:12] \ Bits [11:8]	0,1	2,3	4,5	6,7	8,9	A,B	C,D	E,F	
0	LSLS i5				LSRS i5				LSLS/LSRS/ASRS r1, r2, #imm5
1	ASRS i5				ADDS r	SUBS r	ADDS i3	SUBS i3	ADDS/SUBS r1, r2, r3
2	MOVS i8				CMP i8				ADDS/SUBS r1, r2, #imm3
3	ADDS i8				SUBS i8				MOVS/CMP/ADDS/SUBS r1, #imm8
4	[B]				LDR pc				LDR r1, [pc, #4*imm8] (*aka* LDR r1, =const)
5	STR r	STRH r	STRB r	LDRSB r	LDR r	LDRH r	LDRB r	LDRSH r	STRx/LDRx r1, [r2, r3]
6	STR i5				LDR i5				STRx/LDRx r1, [r2, #s*imm5]
7	STRB i5				LDRB i5				
8	STRH i5				LDRH i5				
9	STR sp				LDR sp				STR/LDR r1, [sp, #4*imm8]
A	ADD pc				ADD sp				ADD r1, pc, #4*imm8 (*aka* ADR r1, label)
B	[C]								ADD r1, sp, #4*imm8
C	STM				LDM				STM/LDM r1!, {regs}
D	[D]								
E	B								B 2*disp11
F	[E]								

0×1840 = 0b0001 100 001 000 000

adds r0, r0, r1

Conventions?

? Who decides these encodings?

The instruction set designers! In this case ARM.

- It is *chosen* by humans (i.e. not an eternal truth)
- But, like everything in this course, not *arbitrary*!
- Design constraint: Encode as many useful instructions into 16 bits as possible!

Show how to find this in the datasheet

- `adds` (register)
- `adds` (immediate)

Consequences of Encoding

We know know *why*

- Only the lower 8 registers can be accessed by most instructions.
  - Allocating 3 bits to indicate the register only gives 8 options!
- The assembler language `adds r0, r1, #10` is illegal.
  - We only have 3 bits to allocate to the constant, giving only 8 options!

The Next Instruction

Now the `pc` points to 194 in memory, and contains the instruction

`0x4770 = 0b01000111 0 1110 000`

which decodes to: `bx`.

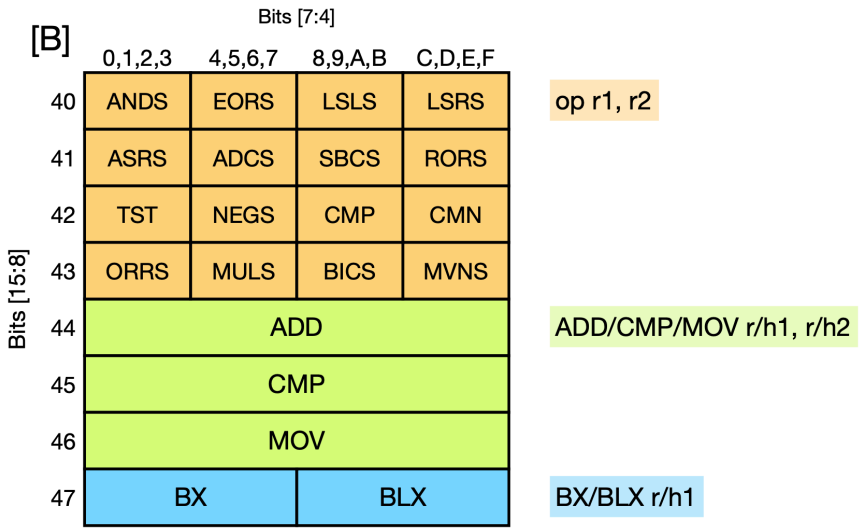
Looking at the instruction reference:

Branch to reg	`bx rm`	*pc* := R*m*	High regs allowed
Branch reg & link	`blx rm`	*lr* := *next*; *pc* := R*m*	

Regs encoded in following bits

`0b1110` = 14, and `r14` is `lr`, so:

`bx` lr



Datasheets: How we know how regs are encoded

This behaviour is not mysterious. It was *designed* by humans. Humans want others to understand. So they write documentation / datasheets. ARM publishes all this in easy-to-search formats!

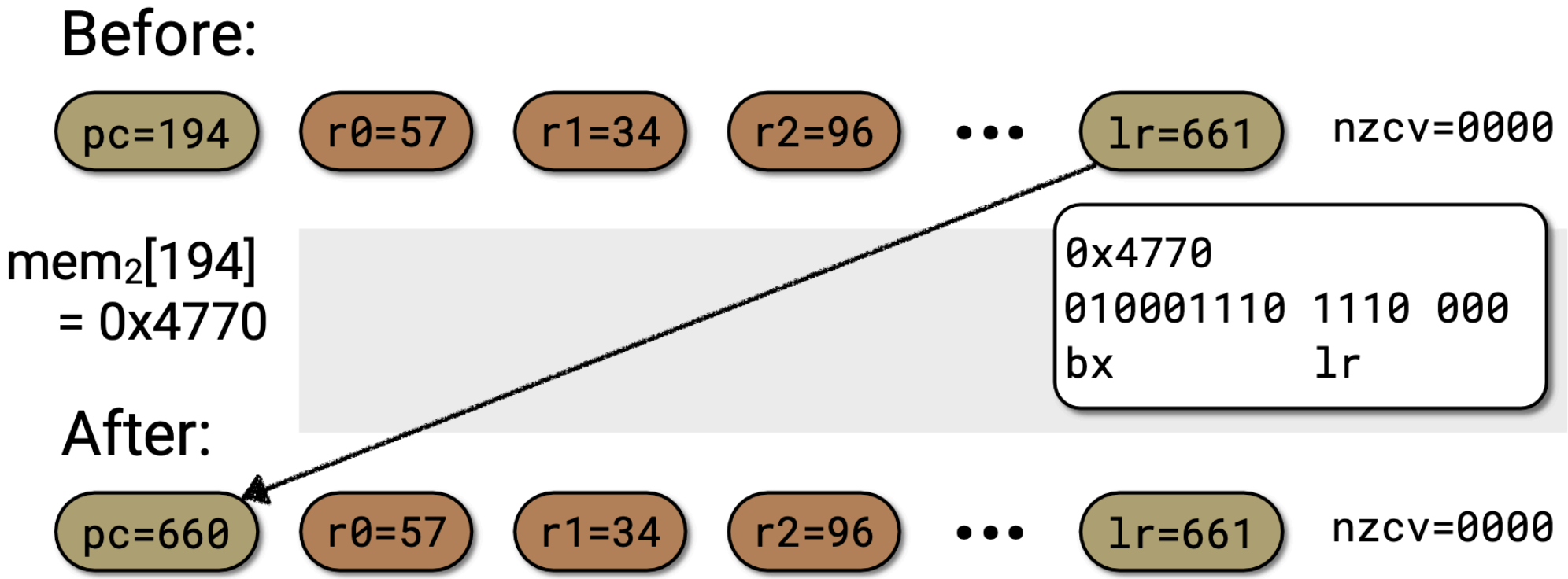
So if you:

- go to README in **gitlab.cs.ox.ac.uk/marilk/digital-systems/**,
- Find the link for "architecture reference manual" under "Resources",
- look at the contents, and find "A5.1 Thumb instruction set encoding",

you will find all information that the rainbow chart was made from.

Follow links further, and you will find A6.7.15 for `bx`.

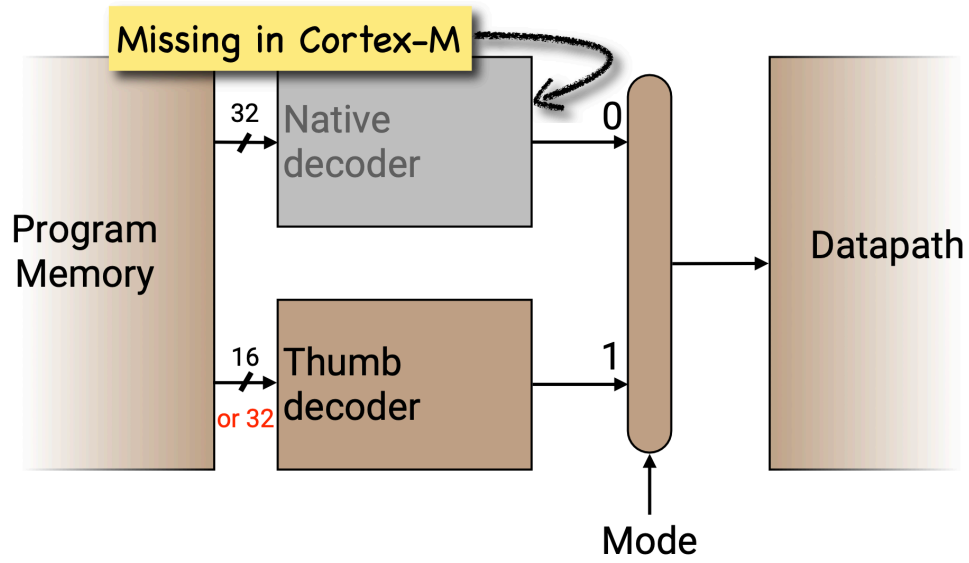
The Effect of `bx lr`



Program Counter

There are multiple encodings for ARM instructions. Big chips can choose.

- "Native" ARM instructions (32 bits)
- "Thumb" instructions (16 bits)
- Thumb-2 extensions (mixture of 16- and 32 bit)



Cortex-M0 only has Thumb instructions, but Cortex-M4 uses ThumbV2.

Subroutines

? How can we make parts of a program reusable?

Store a re-usable sequence of instructions, starting at a known memory location (e.g. 660). Then run program:

- Instr: Store location of next instruction in `lr`.
- Instr: Change the `pc` to known subroutine location.
  - Run multiple instructions in subroutine.
  - Change the `pc` back to `lr`.
- Continue program.

We saw how `bx lr` allows the program to continue.

Subroutines

⚠ **Register values remain the same between subroutines!**

- When jumping into a subroutine, the registers may be in use by the caller.
- Subroutine needs to use registers as well!

⇒ We need conventions on how to use registers to prevent overwriting data that is in use, and how to communicate results of subroutine.

Lab Exercise One

- Practice writing assembly language.
- Your assembly language will be in a subroutine.
- Called from a bigger C program that handles I/O.

Conclusion

❓ How do instructions alter the CPU state?

- Understand the effect of instructions on the state of the CPU.
- Worked example of executing an instruction.

❓ How are instructions encoded electronically?

And how does this affect programming the machine?

💪 Decode machine code

I.e. how to take machine code, and determine what it will do.

❓ How can we make parts of a program reusable?

How do subroutines work?

References / Further reading

- Image of 386 chip. **https://collection.sciencemuseumgroup.org.uk/objects/co523252/intel-386-microprocessor-1985**
- Image of 386 die. Ken Shirriff's Blog. *Reverse engineering the Intel 386 processor's register cell.* **http://www.righto.com/2023/11/reverse-engineering-intel-386.html**